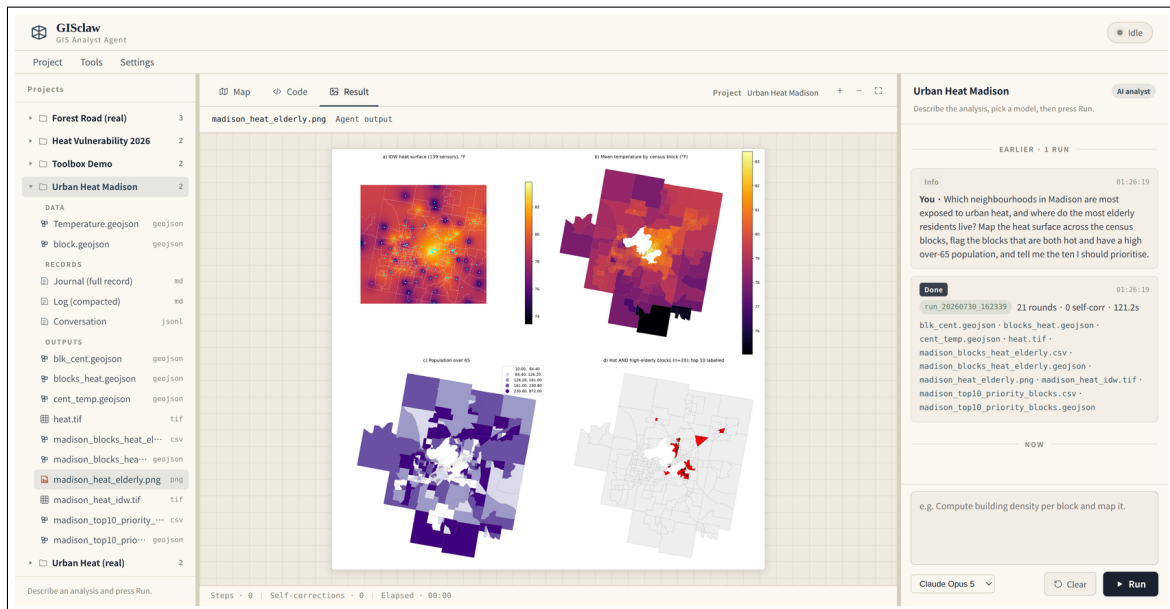


GISclaw

用户手册

说出你要做的空间分析,拿到做完的成果。



GISclaw 是一个地理空间分析自动化工具。你用平常说话的方式提出问题,它规划流程、在你自己的数据上执行,并交回图层、地图、表格,以及一份关于”它是怎么做到的”的书面记录。本手册涵盖安装、日常使用、它留下的记录,以及让跨月项目保持可读的那套机制。

Contents

1	GISclaw 做什么	2
1.1	常见的请求	2
2	安装	2
2.1	环境要求	2
2.2	步骤	2
3	日常使用	3
3.1	新建项目并挂载数据	3
3.2	怎么把需求说清楚	4
3.3	运行时看什么	4
4	一次运行受什么约束	4
5	每个项目留下什么	5
5.1	压缩日志	5
6	不同的输入,不同的反应	6
7	常驻偏好	6
8	Skills	6
9	算子工具箱	7
10	你的数据在哪	8
11	故障排查	8
12	与研究论文的关系	8
13	许可与归属	8

1 GISclaw 做什么

空间分析大多是一串成熟操作的组合:重投影、插值、连接、统计、分级、出图。真正耗时间的是把它们按正确顺序拼起来,错误也藏在这里。

GISclaw 接受的输入,就是领域科学家平时说话的样子。封面那张四联图的全部输入是下面这一句:

Which neighbourhoods in Madison are most exposed to urban heat, and where do the most elderly residents live? Map the heat surface across the census blocks, flag the blocks that are both hot and have a high over-65 population, and tell me the ten I should prioritise.

据此,GISclaw 检查了两个图层,发现坐标系不一致,先做重投影;用 139 个观测点构建 IDW 热力面;对 269 个人口普查街区计算分区均值;把温度与 65 岁以上人口密度归一化后加权合并、排序;绘制图件;写出十个文件。共 21 步,耗时 121 秒。

1.1 常见的请求

你说	它产出
水位超过 2 米时,哪些路段会切断通往医院的通路?	淹没范围、受影响路段、被孤立的汇水区
规划道路 500 米范围内压占了多少保护林?	缓冲区、相交、按保护等级的受影响面积、影响图
哪些片区消防服务覆盖不足?	服务半径、覆盖空白、每个空白区内人口
对比这两个时相的林地变化并给出损失量。	变化栅格、增加/减少/净变化、发散配色地图

2 安装

2.1 环境要求

Docker 与 Docker Compose,约 3 GB 磁盘空间用于镜像,以及一家模型服务商的 API key。无需安装任何 GIS 软件:容器内已包含完整的开源地理空间栈。

2.2 步骤

1. `git clone <[仓库]>`,进入 GISclaw
2. `docker compose up -d`
3. 打开 `http://localhost:8765`
4. **Settings** → **API keys...**,粘贴 key,点击 **Test**

Key 存在哪里。 Key 保存在服务端 `./projects/.gisclaw/settings.json`,文件权限 600。它不会进入镜像、不会到达浏览器,回传给界面的只有 `sk-abc...wxyz` 这样的掩码。若在 CI 或共享主机上使用,`.env` 中的环境变量可作为备用来源。

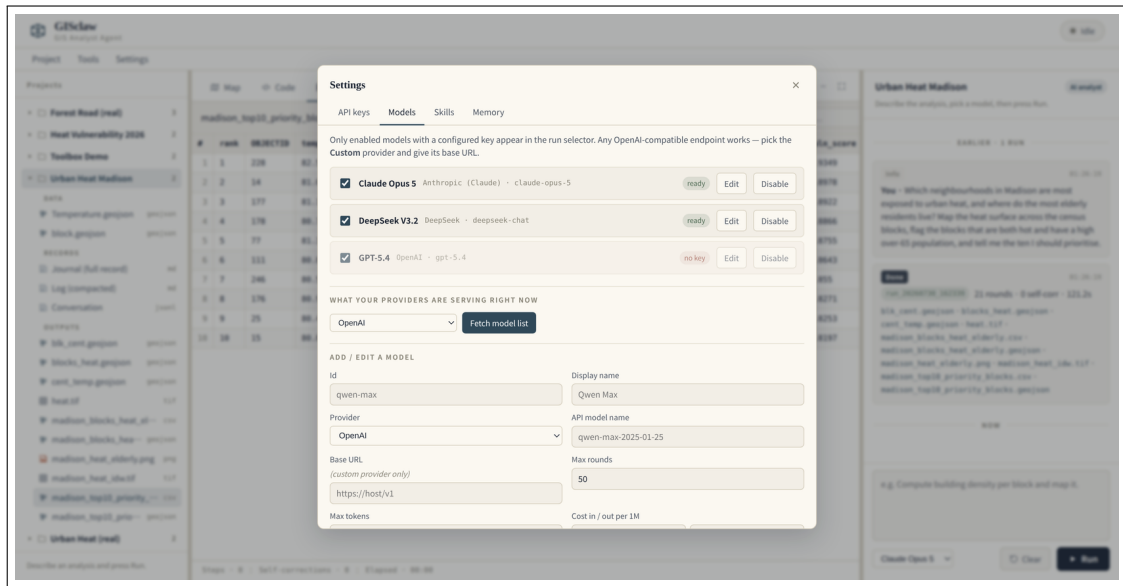


Figure 1: 模型注册表。列表实时从各服务商拉取,新发布的模型无需等待本软件更新;运行下拉里只出现确实配好 key 的模型。

3 日常使用

3.1 新建项目并挂载数据

项目就是一个工作文件夹。用 **Project** → **New project...** 新建,再用 **Project** → **Add data...** 挂载数据——它打开的是以挂载卷为根的服务端文件浏览器。也可以在左侧树里右键项目文件夹。

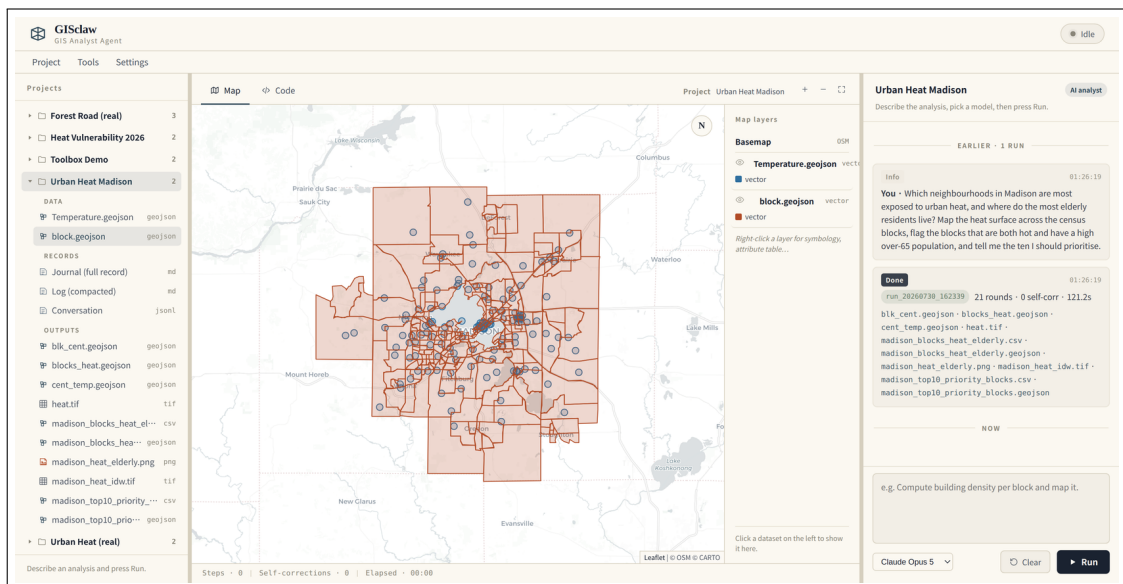


Figure 2: 图层渲染在地图上,左侧是项目树,右侧是对话。右键任意图层可改符号化、查看属性表或缩放。

3.2 怎么把需求说清楚

说明目标、点出真正重要的约束、讲清要什么回来。字段名通常无需提供,agent 会自己读取 schema。

偏弱 分析一下热岛。

可用 用温度点插值,然后按街区统计。

好 把 TemperatureF 点插值成连续面,对每个人口普查街区计算分区均值,把 blocks_meantemp.geojson 和一张 choropleth PNG 存到 pred_results/,并报告最热的五个街区以及有多少街区没有值。

要一个你能核对的数字——最热的五个街区、没有值的数量——是发现”跑完了但结果不对”最省事的办法。

3.3 运行时看什么

Thought、Action、Observation 实时流入对话。生成的代码出现在 **Code** 标签页,图件在 **Result**,表格与文本在 **Table**,矢量或栅格产物自动渲染到地图。

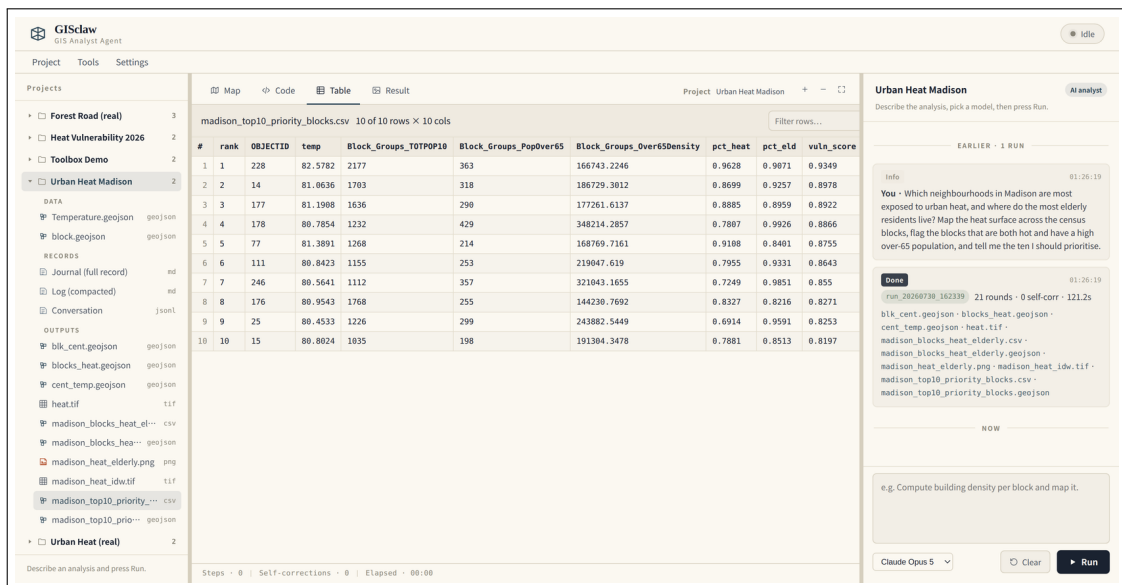


Figure 3: 提问、带产物清单的运行摘要,以及在查看器中打开的结果表。

4 一次运行受什么约束

GISclaw 在持久化 Python 沙箱上跑 ReAct 循环:看数据、推理下一步、写几行代码、读结果、继续。

其操作纪律来自本 agent 在真实多步 GIS 任务上约 1,800 次受控实验,并应用于每一次分析。

1. **先定完整路径再写代码。** 最主要的失败是”为错误的计划写了正确的代码”。
2. **列名从数据里读。** 绝不凭印象假设。
3. **把 CRS 当头等问题。** 投影混杂是常态;距离与面积运算在投影坐标系中进行,且每次连接后打印 null 率。
4. **优先使用确定性算子,而不是手写等价代码。**
5. **禁止静默填补缺失值。** 没有观测的单元保持 null。确需填补时,必须给出理由、数量与标记列,并在结论中
6. **报错要诊断,不要重试。** 真正的错误在 traceback 末尾。

7. 结束前先核验。值域、计数,以及上图字段是否真的有变化。

第 5 条为什么存在。 若一次运行用另外 99 个单元的均值填满了 269 个中的 170 个, 产出的地图看起来完整,而其中三分之二的格局是制造出来的——并且能通过所有自动检查。明确说明只覆盖了 99 个单元,价值更高。

5 每个项目留下什么

```
projects/<项目>/
├── data/                数据源
├── outputs/             输出
├── JOURNAL.md           运行日志: 模型运行记录
├── LOG.md               模型运行日志, 模型运行
├── chat.jsonl           模型运行记录, 模型运行
├── runs/run_<项目>/
│   ├── code.py          模型运行代码
│   ├── trace.jsonl      模型运行 Thought / Action / Observation
│   └── pred_results/    模型运行结果, 模型运行
```

刷新浏览器、重启容器、三个月后再来:对话都会从 chat.jsonl 恢复。点击任意历史运行可回放推理过程并

该信哪一份。 outputs/ 是便于取用的汇总副本,后续运行可能以同名文件覆盖; runs/.../pred_results/ 永不改动,任何一次具体结果都可从那里精确复现。

5.1 压缩日志

长期项目会堆出没人重读的记录。每次运行结束后,GISclaw 用一次模型调用把轨迹压成五个字段——Result、Method、Numbers、Caveats、Carries forward——追加进 LOG.md。注入下一次运行上下文的正



Figure 4: 压缩日志。每一条都由模型依据该次运行的完整轨迹写成。

6 不同的输入,不同的反应

GISclaw 会先判断收到的是哪一类消息,再决定怎么响应。因此它在一件工作的全过程都用得上,而不只是在你已

- **分析请求** —— 启动一次运行。
- **关于项目的提问** —— 做过什么、某个图层里有什么、某个值为什么看着不对 —— 用一次调用从记录里作答
- **想先斟酌的方案** —— 可以讨论。你可以描述一个思路、问它面对现有数据会怎么做、让它比较两种方法的
- **日常对话** —— 就当日常对话处理。
- **超出范围的请求** —— 婉拒,并说明它能处理什么。

讨论一个方法只需一次短调用;真跑一遍可能要几分钟和一美元。先把路线谈拢,往往是通向正确答案更省的那条路。

7 常驻偏好

全局 MEMORY.md 保存适用于一切工作的约定:配色方案、统一采用的分级方法、本地区的投影坐标系、每张交付 **Settings** → **Memory...** 写一次,或在任意消息上悬停点击 **Remember**,它会注入每一次运行。

内容以规则为主。HTML 注释中的文字在注入前会被剥除,因此写给自己的说明不产生任何开销。

8 Skills

一个 skill 是用来固化”某一类分析该怎么做”的目录包:

```
<skill>/
├─ SKILL.md          YAML frontmatter  []
├─ manifest.yaml     []
└─ references/*.md   [],[]
```

加载是渐进的,因此技能库再大也负担得起:一行描述常驻上下文(约 80 token),命中时才载入路由,reference 文件按需逐个打开。

层级	内容	何时加载
目录	名称与一行描述	始终
路由	SKILL.md 正文与文件清单	被打开或被匹配时
深度	单个 reference 文件	某一步需要时

应用内置两个 skill。gis-analysis-discipline 承载第 4 节的纪律,默认常驻;gis-workflow-recipes 收录四类任务的分步模板:插值加分区统计、多准则适宜性叠加、邻近与影响评估、两时相变化检测。

技能包支持以 .zip 或文件夹导入、在应用内编辑、导出分享。编写时请在 frontmatter 中声明 keywords:,服务端据此匹配并预加载合适的 skill。

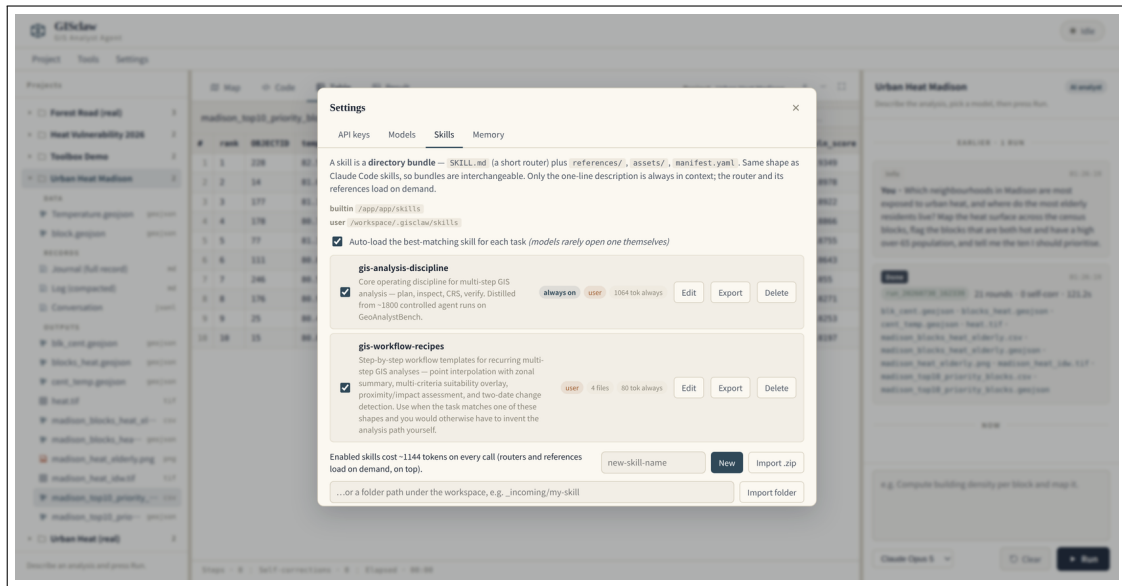


Figure 5: Skills 面板,显示来源、包内文件数,以及每个启用的 skill 增加的 token 开销。

9 算子工具箱

应用内置 28 个确定性的地理处理算子:重投影、缓冲、裁剪、相交、相减、合并、融合、空间与属性连接、分区等。

保留它们有两个实际理由。其一,它们经过测试且正确处理 CRS,agent 会优先调用而不是手写等价代码;其二,当你已经确知要做哪一步时,直接在面板里运行完全不产生 API 花费,结果立即上图。

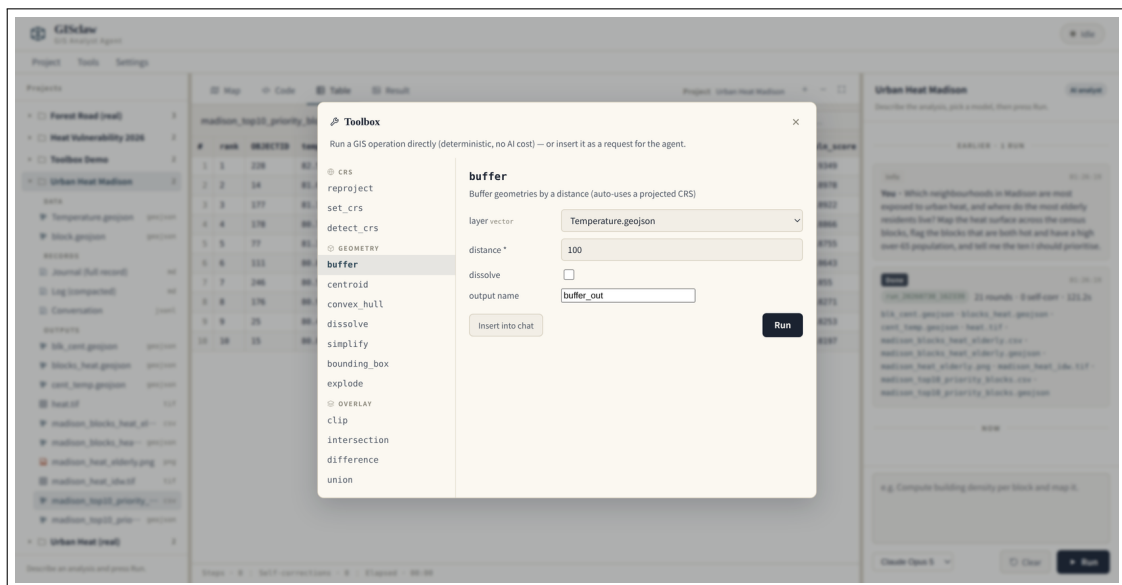


Figure 6: 每个算子都有真实的参数表单。Run 直接确定性执行; Insert into chat 把它作为更大流程中的一步交给 agent。

10 你的数据在哪

分析在容器内针对原生格式进行:Shapefile、GeoTIFF、GeoPackage、CSV、NetCDF。转成 Geo-JSON 或 PNG 只发生在浏览器显示这一层,因为浏览器只能画这些。文件本身留在你的机器上。发送给模型服务商的,是你的提问以及 agent 对数据的观察。

11 故障排查

现象	检查
模型下拉显示 No model configured	没有任何服务商配置了 key。进 Settings → API keys 并点 Test 。
运行刚开始就报 key 错误	所选模型的服务商没有 key;换一个标记为 ready 的模型。
栅格操作在宿主机上失败	栅格请在容器内运行;宿主机 PROJ 数据库损坏会影响 rasterio.warp,矢量不受影响。
更新后界面看起来还是旧的	强制刷新浏览器,并确认控制台报告的 app.js 版本符合预期。
outputs/ 里出现中间文件	每一步 geoprocess 都写入 pred_results/,中间产物会与最终成果一并复制。

服务端日志:docker logs -f gisclaw_app。单次运行日志: projects/< >/runs/<run_id>/run.log。

12 与研究论文的关系

arXiv:2603.26845 描述的系统是为受控实验构建的评测框架。本桌面应用在那套 agent 核心之上重建,此后已有大量分化:界面可用的算子工具箱、带日志与压缩摘要的项目持久化、具备渐进披露的 skill 包、全局偏好 memory,以及在线模型发现。

论文报告的实验结果由研究框架在其冻结 commit 上产生。复现工作应使用那份 artifact。

13 许可与归属

本软件采用 MIT 许可。内置示例数据来自 GeoAnalystBench(Zhang et al., 2025),依 Apache-2.0 分发,并有独立的引用要求;完整的第三方材料清单见 examples/urban-heat-madison/README.md 与 THIRD_PARTY_NOTICES.md。